Octopus CXL Memory Pooling

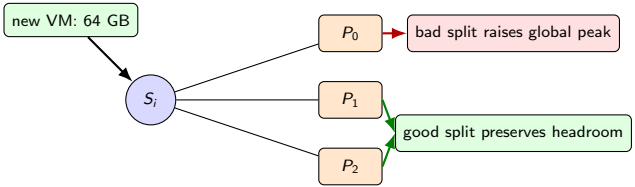
Systems evidence, current failure mode, and RL redesign roadmap

Pulak Mehrotra

March 2026

Roadmap

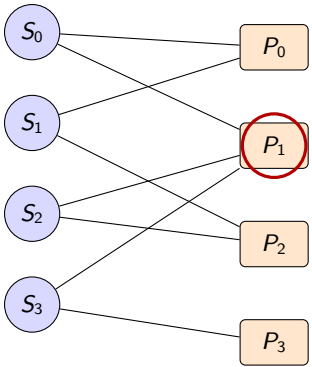
Problem Setting



- Each VM arrival triggers one memory-allocation decision.
- A host can only use its physically connected MPDs.
- There is no cheap migration loop to fix bad placements later.
- We care about the worst MPD peak because every MPD must be provisioned to survive it.

$$\text{Current savings} = 1 - \frac{m \cdot \max_j\{\text{PeakLoad}_j\}}{\text{total pod DRAM}}$$

Octopus Topology Makes Allocation Hard



- Servers and MPDs form a sparse bipartite graph, not a fully connected pool.
- A hot subset of servers can collide on the same small MPD neighborhood.
- A choice that looks good for one server can still overload a shared MPD later.
- So Octopus is harder than "just spread load evenly".

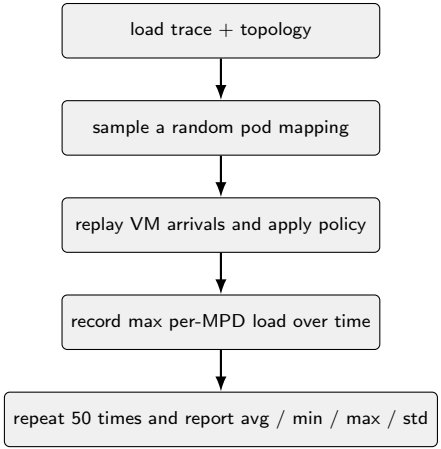
Systems takeaway: topology is part of the problem statement, not a detail of the simulator.

Why Pooling Helps

Missing figure:
../octopus-allocation/docs/overview/v1/figs/peak_to_mean.pdf

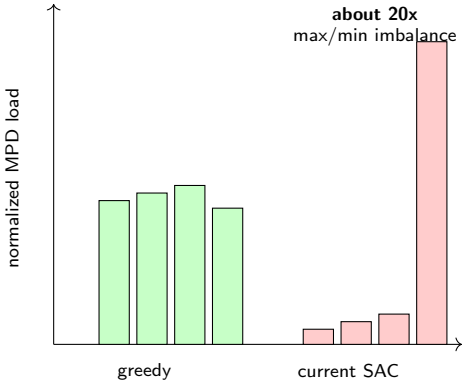
- Azure traces across **10 datacenters** show unsynchronized peaks.
- Cluster peak-to-mean ratio is about **1.13–1.26x**.
- That gap is exactly the headroom memory pooling tries to recover.
- The pooling opportunity is real before we even talk about RL.

Current Implementation



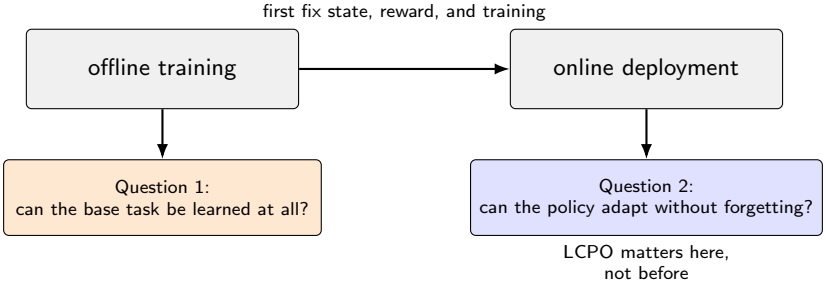
- ‘evaluate.py‘ does not score one cherry-picked topology.
- It repeats evaluation over many random pod mappings.
- The reported number is a distribution of pooling savings, not one run.
- This is a systems metric: required per-MPD capacity under trace replay.

Why Current SAC Fails Right Now

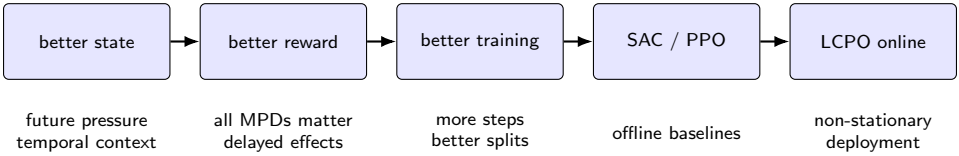


- Current agent: **SAC**, 1M steps, fast configuration.
- The failure is behavioral: load collapses onto a few MPDs.
- Diagnostics point to:
  - sparse reward
  - missing future-pressure features
  - missing temporal features
  - too little training
  - weak train/eval diversification
- This is a **learnability problem** first, not a forgetting problem yet.

Training-Time Learnability vs Deployment-Time Forgetting

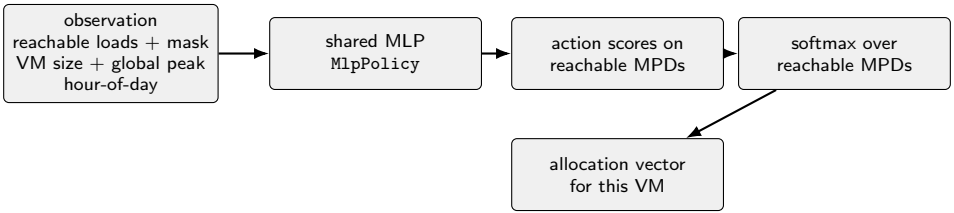


Redesign Roadmap



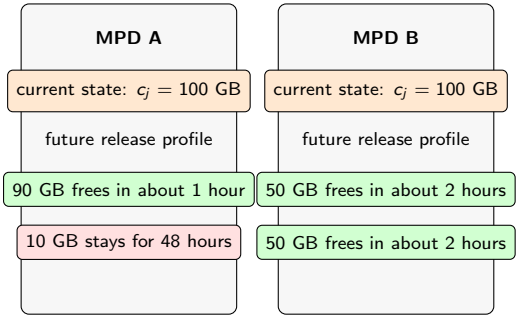
Main message: first make the task learnable, then solve online forgetting.

Current Decision Pipeline in the Code



- The model *does* see each reachable MPD separately.
- It still does not see how each MPD is likely to fill up over time.
- The problem is not missing MPD slots in the input.
- The real issue is that the current state is too instantaneous and the reward is too sparse.

State Space Redesign: Adding More MPD Data



- Current state keeps only one scalar per MPD: c_j = current load.
- So MPD A and MPD B both collapse to the same observed value, $c_j = 100$ GB.
- But their future headroom is very different.
- Add per-MPD features for:
  - short-term release within 1 hour
  - medium-term release within 4 hours
  - sticky load weighted by remaining lifetime
  - VM count as a fragmentation signal
- VM lifetime is estimatable at arrival, so these features are computable.

Sticky load alone is useful, but not sufficient: it tells us how locked an MPD is overall, while the release-horizon features tell us **when** capacity comes back.

Citation: Pond (ASPLOS 2023), paper link

State Space Redesign: Adding Temporal Data

Temporal feature	What it adds
Per-MPD deltas	Whether each MPD is filling or draining
Per-server deltas	Whether local pressure is ramping up
Recent SLO summaries	Whether congestion is getting worse
Hour-of-day context	Where we are in the diurnal cycle

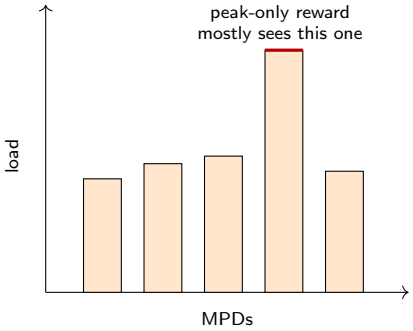
- The current policy mostly sees position, not velocity.
- Two MPDs with the same current load can have opposite near-future trends.
- Temporal features tell the policy whether pressure is rising, stable, or falling.
- Systems intuition: if traces are diurnal, timing information is operationally valuable.

Why We Need More Than One New MPD Feature

Feature	What it adds
Short release	Near-term memory that opens up
Medium release	Headroom that appears a bit later
Sticky load	How trapped the MPD is overall
VM count	Concentrated vs fragmented occupancy

- Sticky load is one summary, not the whole story.
- Release-horizon features preserve timing.
- VM count captures fragmentation that total load can miss.
- The policy needs **how much**, **how soon**, and **how fragmented**.

Why Peak-Only Reward Ignores Most MPDs



- Current reward changes only when the global peak changes.
- If action quality differs among non-peak MPDs, reward can stay unchanged.
- So the network sees all MPDs, but the scalar reward mostly scores one of them.
- Result: the policy can ignore most of the load-distribution structure.

$$r_t = -\frac{\text{peak}_{\text{after}} - \text{peak}_{\text{before}}}{\text{fairShare}}$$

Reward Redesign: New Immediate Reward

Old immediate reward

- mostly controls the current worst MPD
- sparse signal
- weak incentive to spread load

New immediate reward

- peak penalty
- variance penalty
- oversubscribed-MPD penalty

$$R_t = \alpha \cdot \text{PeakTerm}(t) - \beta \cdot \text{VarianceTerm}(t) - \gamma \cdot \text{OverCapMPD}(t)$$

$$\text{PeakTerm}(t) = 1 - \frac{\max_j \{c_j(t)\}}{\text{cap}_{\max}}$$

$$\text{VarianceTerm}(t) = \text{Var}(c(t))$$

$$\text{OverCapMPD}(t) = \sum_j \max\{0,\ c_j(t) - \text{cap}_j\}$$

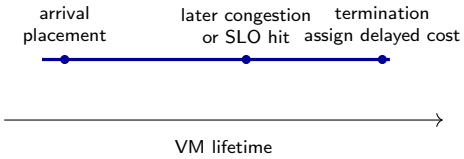
- This is easier to optimize than peak-only reward because all three terms move during training.
- The oversubscribed-MPD term is a direct penalty on capacity violations, not a generic "SLO" abstraction.

Reward Redesign: Why Each Term Matters

Term	What it measures	Why we need it
Peak term	current worst MPD load	keeps the reward aligned with the true capacity objective
Variance term	imbalance across MPDs	gives dense feedback so non-peak MPDs matter too
Over-cap term	total oversubscribed memory across MPDs	makes capacity violations a hard penalty rather than an afterthought

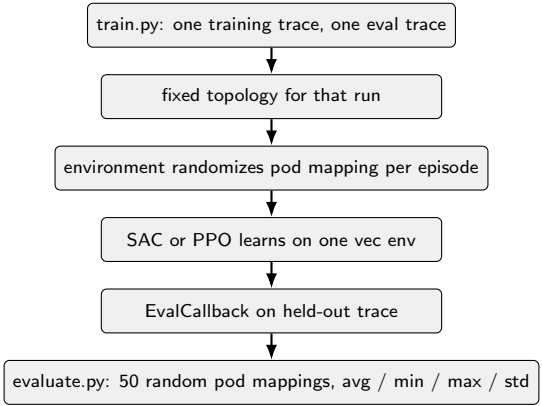
- **Peak term only** is not enough:
  - it mostly watches one MPD
  - it can ignore imbalance elsewhere
- **Variance term only** is not enough:
  - perfectly balanced but very high load is still bad
- **Over-cap term** matters because oversubscription is the real operational failure mode.
- Together the reward pushes toward **uniformly low** load, not merely balanced load.

Reward Redesign: Post-Termination Credit



- A bad placement can look harmless immediately.
- The real damage may appear hours later when later VMs arrive.
- Add a termination-time reward for:
  - lifetime SLO impact
  - migration cost if migration is ever allowed
- This is how we address long-horizon credit assignment.

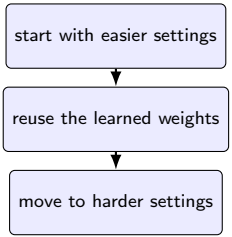
Current Octopus Training and Evaluation Pipeline



Sources: `octopus-allocation/scripts/train.py`, `octopus-allocation/scripts/evaluate.py`

- Current code path is simple and reproducible.
- It already has some randomness through pod assignment.
- But it is still one-trace, one-topology training per run.
- There is no curriculum yet.

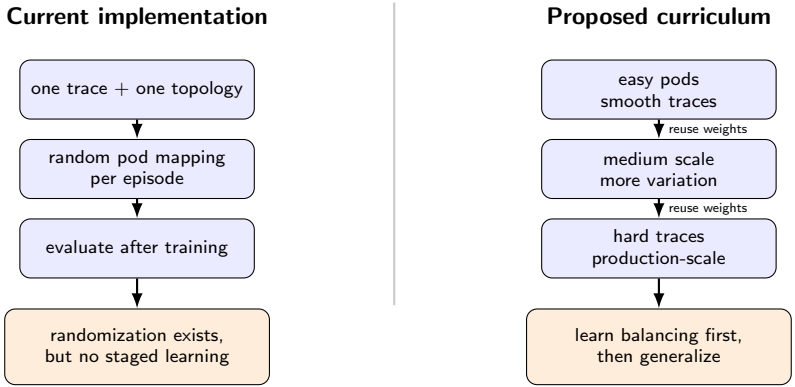
What Is Curriculum Learning?



- Curriculum learning means **training in stages**, not jumping directly to the hardest setting.
- Each stage teaches a simpler version of the same task.
- The model carries its weights forward as the task gets harder.
- Goal: learn the balancing pattern first, then learn large-scale generalization.

Sources: roadmap notes and conversation summary in the slide context folder

Current Pipeline vs Proposed Curriculum



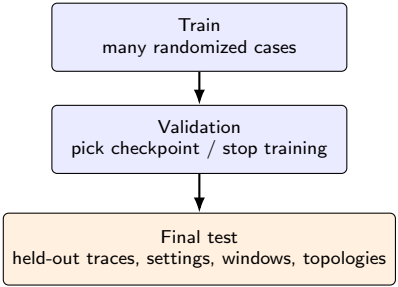
Left: current code path from `train.py` and `evaluate.py`. Right: proposed staged training from the roadmap/context notes.

Domain Randomization and Train/Eval Split

Factor	Train	Eval / test
Datacenter trace	randomized	held out
Pod mapping	randomized	repeated draws
CXL fraction	sweep	held-out settings
Time window	randomized	unseen windows
Topology variant	randomized	held-out cases

- Randomization should be a design choice, not an accident.
- Validation should decide convergence.
- Final test traces should stay untouched until the end.
- This is how we make the evaluation legible to a systems audience.

How To Read This Train / Eval Split



- During training, we **intentionally vary** the environment so the policy learns a rule, not one instance.
- Validation is the tuning loop: it decides whether learning is improving.
- Final test is a firewall: those traces and settings stay untouched until the end.
- Repeated random pod mappings in test give a distribution, not one lucky number.

This is the evaluation discipline we want the audience to trust.

Current code already supports repeated pod remapping in `evaluate.py`; the broader split is the proposed experimental protocol.

Exact Technical Implementation of This Split

Component	Implementation change
Train config	Replace single-trace args with a train manifest: traces, topology IDs, CXL fractions, and time windows.
Env creation	In `_make_env()`, sample one case from that manifest at episode reset.
Validation	Keep a separate fixed validation manifest and point `EvalCallback` only to it.
Best model	Save checkpoints by validation reward, then freeze the best one for test.
Final test	Make `evaluate.py` iterate over a held-out test manifest with the current repeated pod remaps.
Reporting	Report mean / min / max / std per case and then aggregate across cases.

- **Minimal code shape:**

Suggested flow

train.py reads a train manifest and randomizes episodes from it.

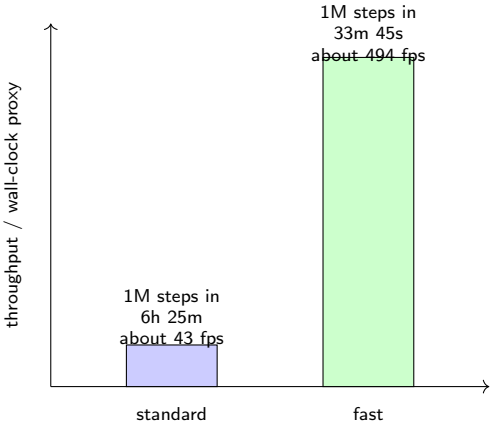
A fixed validation manifest selects the best checkpoint.

evaluate.py runs that checkpoint on a held-out test manifest with 50 pod remaps per case.

- This keeps the current code structure, but makes the split explicit and reproducible.
- It also gives a clean answer when asked: "what exactly was seen during training?"

Based on the current `train.py` / `evaluate.py` structure, with proposed extensions for manifests and held-out suites.

Training Budget Is Part of the Systems Story



- Current logs show that training configuration changes wall-clock cost by an order of magnitude.
- Standard 1M-step SAC run: about **6h 25m**; fast 1M-step SAC run: about **33m 45s**.
- Current SAC training also does limited optimization work relative to collected experience.
- In `--fast`, the code uses `train_freq=16` and `gradient_steps=1`: one gradient update for every 16 env steps.
- That likely improves throughput, but it may also leave the policy under-optimized and hurt generalization.

Timing comes from current training logs; update-frequency details come directly from current `train.py`.

Off-Policy vs On-Policy vs Online RL

Setting	What data it learns from	Main strength	Main risk for Octopus
Off-policy	old + new experience via replay	sample efficient	mixes regimes; can destabilize on drifting traces
On-policy	only fresh rollouts from the current policy	cleaner updates	needs many more environment steps
Online RL	keeps updating after deployment starts	adapts to real drift	can forget old regimes or destabilize behavior

How this maps to our methods

SAC = off-policy offline baseline
PPO = on-policy offline learnability test
LCPO = online adaptation with forgetting control

- The distinction is about **where updates come from**, not just about algorithm names.
- Our roadmap is: first solve the offline task, then decide whether online adaptation is needed.

SAC: What It Is and Why We Started Here

Core idea

Soft Actor-Critic is an **off-policy** method for continuous actions.

It keeps a replay buffer, reuses old experience, and optimizes both reward and exploration.

Useful equation

$$J_\pi = \mathbb{E}\big[Q(s,a) - \alpha \log \pi(a|s)\big]$$

$$y = r + \gamma\big(Q_{\text{targ}}(s',a') - \alpha \log \pi(a'|s')\big)$$

High-value actions are preferred, but the entropy term keeps exploration alive.

SAC: Haarnoja et al., arXiv:1801.01290

- **Fit for Octopus:** our action is a continuous allocation vector.
- **Why start here:** replay makes SAC sample efficient offline.
- **Key risk:** replay mixes old and new regimes, which can destabilize learning on non-stationary traces.
- **Question it answers:** can an off-policy continuous controller learn the allocator at all?

PPO: Simpler Policy-Gradient Baseline

Core idea

Proximal Policy Optimization is an **on-policy** method.

It collects a fresh rollout, computes advantages, and then updates the policy while explicitly limiting how far the policy can move in one step.

Useful equation

$$L^{\text{clip}} = \mathbb{E}\Big[\min\big(r_t A_t,\ \text{clip}(r_t, 1-\epsilon, 1+\epsilon)A_t\big)\Big]$$

$$r_t = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

Take a policy-gradient step, but clip updates so the policy cannot jump too far.

PPO: Schulman et al., arXiv:1707.06347

- **Why it matters:** PPO removes replay-buffer effects and uses fresh rollouts only.
- **Why systems people like it:** cleaner updates make debugging easier.
- **Trade-off:** usually more stable than SAC, but less sample efficient.
- **Question it answers:** if PPO still fails, the bottleneck is probably state / reward / observability, not just SAC instability.

LCPO: Online Adaptation Without Forgetting

Core idea

LCPO is an **on-policy online adaptation** method for non-stationary environments.

It updates on recent data, but constrains the new policy so it does not drift too much on older, different contexts.

Useful equation

$$\max_{\theta} \; L_{\text{recent}}(\theta; B_r) + \lambda H(\pi_\theta)$$

$$\text{s.t.} \quad D_{\text{KL}}\big(\pi_{\theta_{\text{old}}} \,\|\, \pi_\theta; S_c\big) \leq c_{\text{anchor}}$$

Improve on the current workload, but do not drift too far on anchor samples from old contexts.

LCPO: Hamadanian et al., ICLR paper

- **Anchor set:** keep a few old contexts, like morning-load samples while adapting to evening load.
- **Why it matters:** deployment traffic is non-stationary, so online learning can forget earlier regimes.
- **What it adds beyond PPO:** forgetting resistance, not better base-task learning.
- **Question it answers:** once the allocator works, can it adapt online without breaking old behavior?

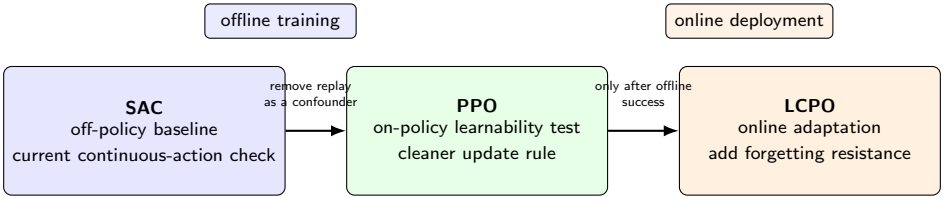
Which Algorithm Solves Which Problem?

SAC	PPO	LCPO
• continuous allocation vector	• same action space	• online non-stationary learning
• sample efficient	• simpler updates	• forgetting resistance
• risk: off-policy instability	• risk: less sample efficient	• risk: more complexity
• role: current baseline	• role: test base learnability	• role: deployment-time method

- This deck is not arguing "LCPO now".
- It is arguing "fix the base task first, then move to deployment-time methods".

SAC: Haarnoja et al., arXiv:1801.01290 PPO: Schulman et al., arXiv:1707.06347 LCPO: Hamadanian et al., ICLR paper

Recommended Order: SAC, Then PPO, Then LCPO



Decision rule: if PPO still fails, fix state / reward / training first. Do not jump to LCPO.

Results We Already Have

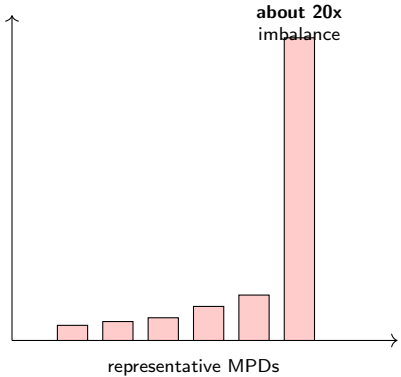
Missing figure:
../octopus-allocation/docs/overview/v1/figs/dem

Already supported

- 10-datacenter trace evidence with clear diurnal structure
- cluster peak-to-mean ratio around 1.13–1.26x
- break-even at about 3% savings
- prior Octopus result of about 16% greedy savings on Acadia-96

This slide combines current files in the repo with older project evidence that should be clearly labeled as such.

Current Measured Limitation: Load Collapse



- The most concrete current failure signal is **about 20x max/min MPD imbalance**.
- For a systems audience, this is easier to reason about than an RL loss curve.
- It says the policy is not yet acting like a load balancer.
- This is the slide that justifies the state and reward redesign.

What Current Training Logs Tell Us

Current-code measurements

- Standard SAC, 1M steps: final eval reward about **-3.17**.
- Fast SAC, 1M steps: final eval reward about **-2.71**.
- Eval reward is still noisy throughout training.
- Current code does *not* log pooling savings during training.

Useful conclusion: current proxy rewards show progress but are still not the systems metric we actually care about.

What the Current Code Path Still Does Not Give Us

- No pooling-savings curve over training time
- No saved greedy-vs-RL comparison table in the new scripted pipeline
- No per-MPD load timeline plot produced automatically from the current run
- No curriculum stages or train/validation/test suite yet

Why this matters

For a systems audience, the next milestone is not just "better reward" but "better observability of the allocator".

Results Dashboard To Add Next

Experiment	What we vary	What we report	What it tells us
State ablation	load-only vs +pressure vs +temporal features	savings, MPD imbalance, oversubscribed MPDs	whether missing observability causes load collapse
Reward ablation	peak-only vs peak+variance vs full reward	savings, over-cap violations, reward stability	whether the reward matches the systems objective
Training ablation	1M vs 10M vs 50M+, with and without curriculum / randomization	validation savings, held-out savings, wall-clock	whether we are under-training or mis-specifying the task
Algorithm comparison	SAC vs PPO offline; LCPO only in online drift setting	held-out performance, forgetting, update cost	whether a new learner helps, or the base task still needs fixing

Minimum dashboard plots

Greedy-vs-RL savings table, per-MPD load timeline, oversubscribed-MPD count, and train / validation / test curves on held-out cases.

Expected Outcomes

- Better state and reward should remove the current load-collapse behavior.
- Better training should improve offline generalization before any online method is introduced.
- PPO should be the clean baseline for asking whether the base task is learnable.
- LCPO should matter only once PPO is already competent and online drift becomes the bottleneck.

Takeaways

- The systems opportunity is real: real traces show headroom, and prior Octopus results already clear break-even comfortably.
- The current RL issue is not "pick a fancier algorithm"; it is that the present task formulation is too hard to learn well.
- The first fixes are better state, better reward, better observability, and more disciplined training.
- The right comparison order is SAC, then PPO, then LCPO.
- Once we have those pieces, we can slim this deck down into a shorter talk very easily.